

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 - APPLICATION BASED QUESTIONS: ANSWERS

Name	K VENKATA VAMSI GOUD
Reg No	192472021
Course Code and Name	DSA0304 - NATURAL LANGUAGE PROCESSING
Assessment	Assessment Tool 1 - Application Based Questions (CO1)

Question 1: Resume Information Extraction System

1. Understanding of the Problem Context

A recruitment company receives thousands of text resumes daily. Manual screening is slow and error-prone, so an automated pipeline is required that converts unstructured resume text into a structured profile and shortlists suitable candidates. The system must satisfy the following requirements:

No.	Requirement	How it is addressed
1	Extract candidate name	Pattern anchored to the 'Name:' label; fallback to the first line of the resume
2	Identify email addresses	Email regex using local part, '@', domain and top-level domain
3	Extract mobile numbers	10-digit Indian number starting with 6-9, optional +91 or 0 prefix
4	Detect technical skills	Whole-word, case-insensitive patterns for Python, Java, SQL, Machine Learning, NLP
5	Extract years of experience	Number followed by 'year(s)' or 'yr(s)', including decimals such as 2.5
6	Structured profile summary	Python dictionary per candidate, printed as a formatted summary
7	Eligibility filter	Experience ≥ 2 years AND Python present in skills

2. Application of Concepts

Field	Regular Expression	Concept Used
Name	<code>^s*Name\s*[:\-\]\s*([A-Za-z][A-Za-z.\-]+?)\s*\$</code>	Anchors (^, \$), capture group, lazy quantifier, re.M and re.I flags
Email	<code>\b[A-Za-z0-9._%+-]+\@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b</code>	Character classes, quantifiers, word boundary

Field	Regular Expression	Concept Used
Mobile	(?<!\d)(?:\+91[[-]]?)?([6-9]\d{9})(?!\\d)	Negative lookbehind/lookahead, non-capturing group, {n} quantifier
Skills	\\bSkill\\b with re.escape and re.I	Word boundaries prevent false matches; escaping handles special characters
Experience	(\\d+(?:\\.\\d+)?)s*\\+?s*(?:years? yrs?)\\b	Optional groups, alternation, decimal support

3. Problem-Solving Approach

- **Step 1:** Compile every pattern once at module level so that it is reused for all resumes (efficient).
- **Step 2:** extract_profile() applies each pattern to the resume text and stores the results in a dictionary.
- **Step 3:** If several experience figures appear, the largest is taken as total experience.
- **Step 4:** is_eligible() checks the two criteria (minimum 2 years and Python skill).
- **Step 5:** print_summary() displays the structured profile; the main block lists only the shortlisted candidates.

4. Python Program

```
import re

# ----- Compiled regular expression patterns -----
NAME_PATTERN = re.compile(r"^\s*Name\s*[:\-\s]*([A-Za-z][A-Za-z.]+?)\s*$", re.I | re.M)
EMAIL_PATTERN = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
MOBILE_PATTERN = re.compile(r"(?<!\d)(?:\+91[[-]]?)?([6-9]\d{9})(?!\\d)")
EXP_PATTERN = re.compile(r"(\\d+(?:\\.\\d+)?)s*\\+?s*(?:years?|yrs?)\\b", re.I)
SKILLS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
SKILL_PATTERNS = {s: re.compile(r"\\b" + re.escape(s) + r"\\b", re.I) for s in SKILLS}

MIN_EXPERIENCE = 2
REQUIRED_SKILL = "Python"

def extract_profile(resume_text):
    """Extract all required fields from one resume using regex."""
    name = NAME_PATTERN.search(resume_text)
    if name:
        name = name.group(1)
    else: # fallback: first non-empty line
        name = resume_text.strip().splitlines()[0].strip()
    emails = EMAIL_PATTERN.findall(resume_text)
    mobiles = MOBILE_PATTERN.findall(resume_text)
    skills = [s for s, p in SKILL_PATTERNS.items() if p.search(resume_text)]
    years = [float(y) for y in EXP_PATTERN.findall(resume_text)]
    return {
        "Name": name,
        "Email": emails[0] if emails else "Not found",
        "Mobile": mobiles[0] if mobiles else "Not found",
    }
```

```

        "Skills": skills,
        "Experience": max(years) if years else 0.0, # largest figure = total experience
    }

def is_eligible(profile):
    return profile["Experience"] >= MIN_EXPERIENCE and REQUIRED_SKILL in profile["Skills"]

def print_summary(profile):
    print("-" * 52)
    print(f'Name      : {profile["Name"]}')
    print(f'Email       : {profile["Email"]}')
    print(f'Mobile      : {profile["Mobile"]}')
    print(f'Skills      : {', '.join(profile["Skills"]) or 'None'}')
    print(f'Experience  : {profile["Experience"]:g} year(s)')
    print(f'Eligible   : {'YES' if is_eligible(profile) else 'NO'}')

resumes = [
    """Name: Arjun Kumar
    Email: arjun.kumar@gmail.com | Mobile: +91 9876543210
    Skills: Python, SQL, Machine Learning, NLP
    Experience: 3 years as Data Analyst""",
    """Name: Priya Sharma
    Contact: priya_sharma92@yahoo.co.in, 8123456789
    Skills: Java, SQL
    Total experience: 5 yrs in backend development""",
    """Name: Rahul Verma
    Email: rahul.verma@outlook.com Phone: 9012345678
    Skills: python, nlp
    Fresher with 1 year of internship experience""",
    """Name: Sneha Reddy
    Email: sneha.reddy@simats.edu | Mobile: 7001234567
    Skills: Python, Java, NLP
    Experience: 2.5 years in AI research""",
]

if __name__ == "__main__":
    profiles = [extract_profile(r) for r in resumes]
    print("===== STRUCTURED CANDIDATE PROFILES =====")
    for p in profiles:
        print_summary(p)
    print("\n===== ELIGIBLE CANDIDATES (>= 2 yrs and Python) =====")
    eligible = [p for p in profiles if is_eligible(p)]
    for i, p in enumerate(eligible, 1):
        print(f'{i}. {p["Name"]:<15} | {p["Email"]:<28} | {p["Mobile"]} | {p["Experience"]:g} yrs')
    print(f"\nTotal resumes: {len(profiles)} | Shortlisted: {len(eligible)}")

```

5. Output

```

===== STRUCTURED CANDIDATE PROFILES =====
-----
Name      : Arjun Kumar
Email     : arjun.kumar@gmail.com
Mobile    : 9876543210
Skills    : Python, SQL, Machine Learning, NLP
Experience : 3 year(s)
Eligible  : YES

```

Name : Priya Sharma
Email : priya_sharma92@yahoo.co.in
Mobile : 8123456789
Skills : Java, SQL
Experience : 5 year(s)
Eligible : NO

Name : Rahul Verma
Email : rahul.verma@outlook.com
Mobile : 9012345678
Skills : Python, NLP
Experience : 1 year(s)
Eligible : NO

Name : Sneha Reddy
Email : sneha.reddy@simats.edu
Mobile : 7001234567
Skills : Python, Java, NLP
Experience : 2.5 year(s)
Eligible : YES

===== ELIGIBLE CANDIDATES (>= 2 yrs and Python) =====

1. Arjun Kumar	arjun.kumar@gmail.com	9876543210 3 yrs
2. Sneha Reddy	sneha.reddy@simats.edu	7001234567 2.5 yrs

Total resumes: 4 | Shortlisted: 2

6. Justification and Clarity of Result

Arjun Kumar (3 years, Python) and Sneha Reddy (2.5 years, Python) satisfy both conditions and are shortlisted. Priya Sharma has 5 years of experience but no Python skill, and Rahul Verma has Python but only 1 year of experience, so both are rejected. The lowercase skills 'python, nlp' in Rahul's resume were detected correctly because of the case-insensitive flag, and the decimal value 2.5 and the '+91' prefix were handled correctly. These test cases cover eligible, ineligible-by-skill and ineligible-by-experience candidates, which confirms the correctness of the logic.

Question 2: Product Search System

1. Understanding of the Problem Context

An e-commerce company wants flexible keyword matching so that customers find products even when they type incomplete or differently cased keywords. The system must support four search types, ignore case, list the matching products and report the number of matches per search.

No.	Requirement	How it is addressed
1	Exact keyword search	Whole-word match using <code>\bkeyword\b</code>
2	Prefix search	A word that begins with the keyword: <code>\bkeyword\w*</code>
3	Suffix search	A word that ends with the keyword: <code>\w*keyword\b</code>
4	Partial keyword search	Keyword anywhere inside the product name
5	Case-insensitive search	<code>re.IGNORECASE</code> flag on every pattern
6	Display matching products	List comprehension using <code>pattern.search()</code>
7	Report of match counts	Table of keyword, search type and number of matches

2. Application of Concepts

Search Type	Pattern	Concept Used
Exact	<code>\bkeyword\b</code>	Word boundaries ensure 'phone' does not match 'Smartphone' or 'Headphones'
Prefix	<code>\bkeyword\w*</code>	Boundary at the start, then zero or more word characters
Suffix	<code>\w*keyword\b</code>	Zero or more word characters, then boundary at the end
Partial	<code>keyword</code>	Plain substring search with <code>re.search()</code>
All types	<code>re.escape() + re.IGNORECASE</code>	Safe handling of special characters and case differences

3. Problem-Solving Approach

- **Step 1:** `build_pattern()` takes the keyword and search mode and returns one compiled, case-insensitive pattern, so there is no repeated if-else logic.
- **Step 2:** `search()` validates the keyword (empty input raises an error) and filters the product list.
- **Step 3:** Every search result is stored as (keyword, mode, count).
- **Step 4:** After all searches, a summary report prints the number of matches for each search and the total number of searches.

4. Python Program

```
import re
```

```

products = [
    "Apple iPhone 15", "Samsung Galaxy Phone", "Wireless Headphones",
    "Bluetooth Speaker", "Phone Case", "Smart Watch", "Laptop Bag",
    "Gaming Laptop", "Headphone Stand", "USB Charger", "Wireless Mouse",
    "Phone Stand", "Smartphone Cover", "Laptop Sleeve", "Speaker Stand",
]

def build_pattern(keyword, mode):
    """Return a case-insensitive compiled regex for the chosen search mode."""
    k = re.escape(keyword.strip())      # escape special characters safely
    patterns = {
        "exact": rf"\b{k}\b",          # whole word equals keyword
        "prefix": rf"\b{k}w*",         # a word BEGINS with keyword
        "suffix": rf"w*{k}\b",         # a word ENDS with keyword
        "partial": rf"{k}",            # keyword anywhere in the name
    }
    return re.compile(patterns[mode], re.IGNORECASE)

def search(keyword, mode):
    if not keyword.strip():
        raise ValueError("Search keyword cannot be empty")
    pattern = build_pattern(keyword, mode)
    return [p for p in products if pattern.search(p)]

queries = [
    ("phone", "exact"),
    ("PHONE", "prefix"),
    ("top", "suffix"),
    ("stand", "partial"),
    ("wire", "prefix"),
    ("less", "suffix"),
    ("lap", "partial"),
    ("tablet", "exact"),
]

if __name__ == "__main__":
    report = []
    for keyword, mode in queries:
        results = search(keyword, mode)
        report.append((keyword, mode, len(results)))
        print(f"Search: '{keyword}' [{mode.upper()}]")
        if results:
            for r in results:
                print(f"  - {r}")
        else:
            print("  No matching products found")
        print()
    print("===== SEARCH REPORT =====")
    print(f"{'Keyword':<10}{'Type':<10}{'Matches':>8}")
    for k, m, c in report:
        print(f"{k:<10}{m:<10}{c:>8}")
    print(f"Total searches: {len(report)}")

```

5. Output

Search: 'phone' [EXACT]
- Samsung Galaxy Phone
- Phone Case
- Phone Stand

Search: 'PHONE' [PREFIX]
- Samsung Galaxy Phone
- Phone Case
- Phone Stand

Search: 'top' [SUFFIX]
- Laptop Bag
- Gaming Laptop
- Laptop Sleeve

Search: 'stand' [PARTIAL]
- Headphone Stand
- Phone Stand
- Speaker Stand

Search: 'wire' [PREFIX]
- Wireless Headphones
- Wireless Mouse

Search: 'less' [SUFFIX]
- Wireless Headphones
- Wireless Mouse

Search: 'lap' [PARTIAL]
- Laptop Bag
- Gaming Laptop
- Laptop Sleeve

Search: 'tablet' [EXACT]
No matching products found

===== SEARCH REPORT =====

Keyword	Type	Matches
phone	exact	3
PHONE	prefix	3
top	suffix	3
stand	partial	3
wire	prefix	2
less	suffix	2
lap	partial	3
tablet	exact	0
Total searches: 8		

6. Justification and Clarity of Result

The exact search for 'phone' returns only the three products in which 'Phone' is a separate word and correctly excludes 'Smartphone Cover' and 'Wireless Headphones'. The upper-case keyword 'PHONE' returned the same products as the lower-case version, which proves case-

insensitivity. The prefix search 'wire' and the suffix search 'less' both return the Wireless products, the suffix search 'top' finds the Laptop items, and the partial search 'stand' finds all three stands. The search 'tablet' returns zero matches and displays a proper message instead of failing. The report counts agree with the listed products for every search.

Question 3: University Registration Validation System

1. Understanding of the Problem Context

Before a student can enrol in a course, the portal must verify that the submitted data is in the correct format. Each field must be validated individually with a clear message, and a final status must show whether registration succeeds. Registration is successful only when all five fields are valid. The following formats are assumed for this implementation:

Field	Accepted Format	Example
Register Number	9 digits starting with 19	192472021
Institutional Email	Valid e-mail with domain @simats.edu.in or @saveetha.com	vamsi.goud@simats.edu.in
Course Code	3 capital letters followed by 4 digits	DSA0304
Semester	A number from 1 to 8, optionally prefixed by 'Semester'	Semester 3
Mobile Number	10 digits starting with 6-9, optional +91	+91 9876543210

2. Application of Concepts

Field	Regular Expression	Concept Used
Register Number	<code>^19\d{7}\$</code>	Anchors, fixed prefix, exact repetition
Email	<code>^[A-Za-z0-9._%+-]+@(simats\.edu\.in saveetha\.com)\$</code>	Character class, alternation, escaped dots
Course Code	<code>^[A-Z]{3}\d{4}\$</code>	Uppercase range, exact repetition
Semester	<code>^(?:Semester\s)?[1-8]\$</code>	Optional non-capturing group, range [1-8]
Mobile	<code>^(?:\+91\s)?[6-9]\d{9}\$</code>	Optional country code, digit range

All patterns are used with `fullmatch()` and start/end anchors so that the complete string must follow the format and not just a part of it.

3. Problem-Solving Approach

- **Step 1:** All patterns and their error hints are kept in one RULES dictionary, so adding or changing a rule needs no change to the logic (modular design).
- **Step 2:** `validate_student()` loops through the rules and returns the status of every field.
- **Step 3:** `report()` prints a VALID or INVALID message with a correction hint for each field.
- **Step 4:** The registration is marked SUCCESSFUL only if all fields are valid; otherwise the failing fields are listed.
- **Step 5:** Empty input is treated as invalid, and a final summary counts successful and failed registrations.

4. Python Program

```
import re

RULES = {
    "Register Number": (re.compile(r"^19\d{7}$"),
        "Must be 9 digits starting with 19 (e.g., 192472021)"),
    "Email": (re.compile(r"^[A-Za-z0-9._%+-]+@(simats\.edu\.in|saveetha\.com)$", re.I),
        "Must use institutional domain @simats.edu.in or @saveetha.com"),
    "Course Code": (re.compile(r"^[A-Z]{3}\d{4}$"),
        "Must be 3 capital letters + 4 digits (e.g., DSA0304)"),
    "Semester": (re.compile(r"^(?:Semester\s)?[1-8]$", re.I),
        "Must be a number 1-8 (e.g., 3 or Semester 3)"),
    "Mobile": (re.compile(r"^(?:\+91[\s-])?[6-9]\d{9}$"),
        "Must be 10 digits starting with 6-9 (optional +91)"),
}

def validate_student(student):
    results = {}
    for field, (pattern, hint) in RULES.items():
        value = str(student.get(field, "")).strip()
        ok = bool(pattern.fullmatch(value)) if value else False
        results[field] = (ok, value, hint)
    return results

def report(student):
    print(f"\nStudent: {student.get('Name', 'Unknown')}")
    results = validate_student(student)
    for field, (ok, value, hint) in results.items():
        msg = "VALID" if ok else f"INVALID -> {hint}"
        print(f" {field}<16>: {value or '(empty)':<24> {msg}")
    success = all(ok for ok, _, _ in results.values())
    failed = [f for f, (ok, _, _) in results.items() if not ok]
    print(" REGISTRATION STATUS: " + ("SUCCESSFUL" if success
        else "FAILED (fix: " + ", ".join(failed) + ")"))
    return success

students = [
    {"Name": "K Venkata Vamsi Goud", "Register Number": "192472021",
    "Email": "vamsi.goud@simats.edu.in", "Course Code": "DSA0304",
    "Semester": "Semester 3", "Mobile": "+91 9876543210"},
    {"Name": "Anita Rao", "Register Number": "29247202",
    "Email": "anita@gmail.com", "Course Code": "dsa304",
    "Semester": "9", "Mobile": "5123456789"},
    {"Name": "Karthik S", "Register Number": "192471005",
    "Email": "karthik.s@saveetha.com", "Course Code": "CSA0101",
    "Semester": "5", "Mobile": "98765"},
]

if __name__ == "__main__":
    outcomes = [report(s) for s in students]
    print("\n===== FINAL SUMMARY =====")
    print(f"Total: {len(outcomes)} | Successful: {sum(outcomes)} | Failed: {len(outcomes)-sum(outcomes)}")
```

5. Output

```
Student: K Venkata Vamsi Goud
Register Number : 192472021          VALID
Email      : vamsi.goud@simats.edu.in VALID
Course Code : DSA0304                VALID
Semester   : Semester 3              VALID
Mobile     : +91 9876543210          VALID
REGISTRATION STATUS: SUCCESSFUL

Student: Anita Rao
Register Number : 29247202          INVALID -> Must be 9 digits starting with 19 (e.g., 192472021)
Email      : anita@gmail.com        INVALID -> Must use institutional domain @simats.edu.in or
@saveetha.com
Course Code : dsa304                INVALID -> Must be 3 capital letters + 4 digits (e.g., DSA0304)
Semester   : 9                      INVALID -> Must be a number 1-8 (e.g., 3 or Semester 3)
Mobile     : 5123456789             INVALID -> Must be 10 digits starting with 6-9 (optional +91)
REGISTRATION STATUS: FAILED (fix: Register Number, Email, Course Code, Semester, Mobile)

Student: Karthik S
Register Number : 192471005          VALID
Email      : karthik.s@saveetha.com VALID
Course Code : CSA0101                VALID
Semester   : 5                      VALID
Mobile     : 98765                  INVALID -> Must be 10 digits starting with 6-9 (optional +91)
REGISTRATION STATUS: FAILED (fix: Mobile)

===== FINAL SUMMARY =====
Total: 3 | Successful: 1 | Failed: 2
```

6. Justification and Clarity of Result

Three test cases were used. The first student has correctly formatted data in all fields, so the registration is successful. The second student has errors in every field (8-digit register number, non-institutional email, lower-case course code, semester 9, mobile number starting with 5), and all five are rejected with specific messages. The third student has only one error, a short mobile number, and the report identifies exactly that field. This confirms that each regular expression accepts valid input, rejects invalid input, and that the final status depends on all fields together.

Conclusion

Regular expressions were applied to three practical NLP tasks: information extraction (anchors, groups, lookarounds), pattern matching (word boundaries, prefix, suffix, case-insensitive flag) and input validation (anchored patterns with fullmatch). The programs are modular, tested with valid and invalid cases, handle errors, and produce clear reports.